目录

基于 CNN 的校园课堂智能考勤系统 — 概要设计文档 **2**
1. 系统架构设计 2
1.1 总体架构 2
1.2 技术选型说明 2
2. 模块划分与职责 3
2.1 后端模块 3
2.2 前端模块 4
2.3 AI 推理模块详细职责 5
3. 数据库设计 5
3.1 ER 图（概念模型） 5
3.2 数据表设计 5
4. 接口设计概要 8
4.1 统一响应格式 8
4.2 接口列表 8
5. AI 模型设计 9
5.1 整体推理流程 9
5.2 迁移学习方案 10
5.3 数据集方案 10
6. 关键设计决策 11
7. 数据库索引设计 11
8. 部署架构设计 12
8.1 开发环境 12
8.2 生产/演示环境（单机部署） 12
8.3 关键配置参数 12
9. WebSocket 通信协议设计 13
9.1 连接建立 13
9.2 消息格式（JSON） 13
9.3 帧率控制策略 14
10. 错误处理策略 15
10.1 HTTP 错误码规范 15
10.2 AI 模块异常处理 15
10.3 统一异常响应结构 15

基于 CNN 的校园课堂智能考勤系统 — 概要设计文档

版本: v1.0
日期: 2026-04-16
依据: 需求分析文档 v1.0

---

1. 系统架构设计

1.1 总体架构

系统采用**前后端分离 + AI 推理服务**的三层架构:

```
                    前端层 (Browser)
          Vue.js 3 + Element Plus + ECharts
    学生管理 / 考勤任务 / 报表统计 / 系统管理

                    HTTP/WebSocket (JSON)

                  后端应用层 (Flask)

   用户认证      学生档案      考勤任务      报表
  Blueprint    Blueprint    Blueprint    Blueprint


                 AI推理模块 (独立服务)
     OpenCV人脸检测 → CNN特征提取 → 余弦相似度匹配


                    SQLAlchemy ORM

                   数据持久层
        MySQL 8.0            本地文件系统
   (结构化业务数据)        (人脸图片/模型权重)
```

1.2 技术选型说明

层次	技术	选型理由
前端框架	Vue.js 3 + Vue Router + Pinia	组件化开发，生态成熟，任务书指定
UI 组件库	Element Plus	后台管理系统首选，表单/表格组件完善
数据可视化	ECharts	考勤报表图表展示
后端框架	Flask 3.x	轻量灵活，与 Python AI 生态无缝集成
数据库	MySQL 8.0	任务书指定，关系型数据适合业务数据
ORM	SQLAlchemy 2.x	安全防注入，模型映射清晰
人脸检测	OpenCV + MTCNN	多级联检测，准确率高于 Haar Cascade
特征提取	VGG16 / ResNet50（PyTorch）	任务书指定对比两种模型
认证	JWT（Flask-JWT-Extended）	无状态，前后端分离适配
跨域	Flask-CORS	开发阶段解决跨域问题
接口文档	Flask-RESTx（Swagger）	自动生成 API 文档

---

2. 模块划分与职责

2.1 后端模块

```
backend/
  app/
      __init__.py          # Flask 工厂函数，注册 Blueprint
      config.py            # 配置（开发/生产）
      models/              # SQLAlchemy 数据模型
          user.py          # 用户模型
          student.py       # 学生模型
          course.py        # 课程模型
          attendance.py    # 考勤记录模型
      api/                 # RESTful 接口 Blueprint
          auth.py          # 认证接口 /api/auth
          students.py      # 学生管理接口 /api/students
          faces.py         # 人脸数据接口 /api/faces
          attendance.py    # 考勤任务接口 /api/attendance
          reports.py       # 报表接口 /api/reports
      services/            # 业务逻辑层
          auth_service.py
          student_service.py
          attendance_service.py
          report_service.py
```

```
    ai/                  # AI推理模块
        face_detector.py # MTCNN 人脸检测
        face_recognizer.py # CNN 特征提取+匹配
        preprocessor.py  # OpenCV 预处理
        model_manager.py # 模型加载与版本管理
  model/                  # 训练相关
    train.py             # 训练脚本 (VGG16/ResNet50)
    evaluate.py          # 评估脚本
    checkpoints/         # 模型权重文件
  dataset/                # 人脸数据集
     faces/
         {student_id}/    # 每个学生的人脸图片
```

2.2 前端模块

```
frontend/
  src/
    main.js
    App.vue
    router/index.js      # 路由配置 + 路由守卫
    stores/              # Pinia 状态管理
        auth.js          # 用户认证状态
        attendance.js    # 考勤任务状态
    api/                 # Axios 接口封装
        request.js       # Axios 实例 + 拦截器
        auth.js
        students.js
        faces.js
        attendance.js
        reports.js
    views/               # 页面组件
        Login.vue
        Dashboard.vue    # 首页数据概览
        students/
            StudentList.vue
            StudentForm.vue
        faces/
            FaceCapture.vue  # 人脸采集页面
        attendance/
```

```
            AttendanceTask.vue  # 发起/监控考勤
            AttendanceList.vue  # 考勤记录查询
        reports/
            ReportPage.vue
```

2.3 AI 推理模块详细职责

子模块	职责	核心技术
`face_detector.py`	输入图像帧，输出人脸区域 bounding box 列表	MTCNN
`preprocessor.py`	裁剪、缩放至 224×224、归一化	OpenCV
`face_recognizer.py`	输入人脸图片，输出 512 维特征向量；与数据库特征余弦匹配得出身份	VGG16 / ResNet50
`model_manager.py`	加载/切换模型权重，维护特征数据库（学号 → 特征向量的 numpy 映射）	PyTorch

---

3. 数据库设计

3.1 ER 图（概念模型）

```
User    (1:N)    AttendanceTask
Student   (M:N)   Course  (通过 StudentCourse 中间表)
Course   (1:N)    AttendanceTask
AttendanceTask   (1:N)   AttendanceRecord
Student   (1:N)   AttendanceRecord
Student   (1:N)   FaceImage
```

3.2 数据表设计

3.2.1 users（用户表）

字段	类型	约束	说明
id	INT	PK, AUTO_INCREMENT	用户 ID
username	VARCHAR(50)	UNIQUE, NOT NULL	登录账号
password_hash	VARCHAR(128)	NOT NULL	bcrypt 加密密码
role	ENUM(‘admin’,‘teacher’,‘counselor’,‘student’)	NOT NULL	用户角色
real_name	VARCHAR(50)	NOT NULL	真实姓名
email	VARCHAR(100)		邮箱（可选）
is_active	TINYINT(1)	DEFAULT 1	账号是否启用

字段	类型	约束	说明
created_at	DATETIME	DEFAULT NOW()	创建时间
updated_at	DATETIME	ON UPDATE NOW()	更新时间

3.2.2 students（学生表）

字段	类型	约束	说明
id	INT	PK, AUTO_INCREMENT	内部 ID
student_id	VARCHAR(20)	UNIQUE, NOT NULL	学号
name	VARCHAR(50)	NOT NULL	姓名
class_name	VARCHAR(50)	NOT NULL	班级
department	VARCHAR(100)		院系
user_id	INT	FK→users.id, NULL	关联系统账号（可选）
face_registered	TINYINT(1)	DEFAULT 0	是否已注册人脸
face_count	INT	DEFAULT 0	已录入人脸图片数量
created_at	DATETIME	DEFAULT NOW()	

3.2.3 courses（课程表）

字段	类型	约束	说明
id	INT	PK, AUTO_INCREMENT	
course_name	VARCHAR(100)	NOT NULL	课程名称
course_code	VARCHAR(20)	UNIQUE	课程编号
teacher_id	INT	FK→users.id	授课教师
classroom	VARCHAR(50)		教室
schedule	VARCHAR(200)		上课时间描述
created_at	DATETIME	DEFAULT NOW()	

3.2.4 student_courses（选课中间表）

字段	类型	约束	说明
id	INT	PK, AUTO_INCREMENT	
student_id	INT	FK→students.id	
course_id	INT	FK→courses.id	
semester	VARCHAR(20)		学期（如：2025-2026-2）
UNIQUE	(student_id, course_id, semester)		防重复选课

3.2.5 attendance_tasks（考勤任务表）

字段	类型	约束	说明
id	INT	PK, AUTO_INCREMENT	
course_id	INT	FK→courses.id	所属课程
teacher_id	INT	FK→users.id	发起教师
task_date	DATE	NOT NULL	考勤日期
start_time	DATETIME		开始时间
end_time	DATETIME		结束时间
status	ENUM(‘pending’,‘running’,‘finished’)	DEFAULT ‘pending’	任务状态
total_students	INT	DEFAULT 0	应到人数
present_count	INT	DEFAULT 0	实到人数
model_version	VARCHAR(50)		使用的模型版本
created_at	DATETIME	DEFAULT NOW()	

3.2.6 attendance_records（考勤记录表）

字段	类型	约束	说明
id	INT	PK, AUTO_INCREMENT	
task_id	INT	FK→attendance_tasks.id	所属考勤任务
student_id	INT	FK→students.id	学生
status	ENUM(‘present’,‘absent’,‘unverified’)	NOT NULL	考勤状态
confidence	FLOAT		识别置信度（0~1）
recognized_at	DATETIME		识别时间
verified_by	INT	FK→users.id, NULL	人工核实操作员
note	VARCHAR(200)		备注
UNIQUE	(task_id, student_id)		同一任务同一学生只记一次

3.2.7 face_images（人脸图片表）

字段	类型	约束	说明
id	INT	PK, AUTO_INCREMENT	
student_id	INT	FK→students.id	所属学生
file_path	VARCHAR(255)	NOT NULL	图片存储路径
is_processed	TINYINT(1)	DEFAULT 0	是否完成预处理
created_at	DATETIME	DEFAULT NOW()	

4. 接口设计概要

4.1 统一响应格式

```
{
  "code": 200,
  "message": "success",
  "data": { ... }
}
```

错误响应:

```
{
  "code": 401,
  "message": "Token expired or invalid",
  "data": null
}
```

4.2 接口列表

认证模块 /api/auth

方法	路径	描述	权限
POST	/api/auth/login	登录，返回 JWT Token	公开
POST	/api/auth/logout	注销	已登录
PUT	/api/auth/password	修改密码	已登录

学生管理 /api/students

方法	路径	描述	权限
GET	/api/students	分页查询学生列表	admin
POST	/api/students	新增学生	admin
GET	/api/students/{id}	查询学生详情	admin/teacher
PUT	/api/students/{id}	更新学生信息	admin
DELETE	/api/students/{id}	删除学生	admin
POST	/api/students/import	批量导入（Excel）	admin

人脸数据 /api/faces

方法	路径	描述	权限
POST	`/api/faces/{student_id}/upload`	上传人脸图片	admin
POST	`/api/faces/{student_id}/capture`	摄像头实时采集	admin
GET	`/api/faces/{student_id}`	查询人脸图片列表	admin
DELETE	`/api/faces/{image_id}`	删除人脸图片	admin
POST	`/api/faces/train`	触发模型训练	admin

考勤任务 `/api/attendance`

方法	路径	描述	权限
POST	`/api/attendance/tasks`	创建考勤任务	teacher/admin
GET	`/api/attendance/tasks`	查询考勤任务列表	teacher/admin/counselor
GET	`/api/attendance/tasks/{id}`	查询任务详情	teacher/admin
POST	`/api/attendance/tasks/{id}/start`	开始考勤（开启摄像头流）	teacher
POST	`/api/attendance/tasks/{id}/stop`	结束考勤	teacher
POST	`/api/attendance/tasks/{id}/recognize`	上传图片触发识别	teacher
GET	`/api/attendance/records`	查询考勤记录（多条件）	teacher/admin/counselor
PUT	`/api/attendance/records/{id}`	人工修正考勤记录	teacher

报表 `/api/reports`

方法	路径	描述	权限
GET	`/api/reports/course/{id}`	课程出勤率统计	teacher/admin/counselor
GET	`/api/reports/student/{id}`	学生考勤历史	all
GET	`/api/reports/export`	导出 Excel 报表	teacher/admin/counselor
GET	`/api/reports/warnings`	缺勤预警列表	counselor/admin

---

5. AI 模型设计

5.1 整体推理流程

输入图像帧

[MTCNN 人脸检测]

```
      输出：人脸 bounding boxes + 关键点

[OpenCV 预处理]
       裁剪人脸区域（加 20px 边距）
       缩放至 224×224
       归一化（ImageNet 均值/标准差）


[CNN 特征提取]（VGG16 / ResNet50 去掉最后分类层）
      输出：512/2048 维特征向量

[L2 归一化]


[余弦相似度匹配]（与内存中所有学生特征向量对比）
      找最近邻

[阈值判断]
       similarity  0.75 → 识别成功，返回 student_id
       0.5  similarity < 0.75 → 标记 unverified
       similarity < 0.5 → 未知人员
```

5.2 迁移学习方案

阶段	操作	说明
加载预训练权重	使用 ImageNet 预训练的 VGG16/ResNet50	`torchvision.models`
冻结底层	冻结前 N 个 Block	保留通用特征，降低过拟合风险
替换分类头	去掉最后 FC 层，改为：FC(512) → BN → ReLU → FC(num_classes)	适配人脸识别分类数
微调训练	解冻全部层，小学习率（1e-4）端到端训练	
特征提取模式	推理时去掉分类头，输出倒数第二层特征	用于余弦相似度匹配

5.3 数据集方案

数据来源	用途	说明
LFW（Labeled Faces in the Wild）	预训练/基准评估	13,000+ 张，1680+ 人
MS-Celeb-1M（子集）	增强泛化能力	选取 1000 人子集
自建校园数据集（脱敏）	Fine-tune 目标域	每人 5 张，不同角度/光线

---

6. 关键设计决策

决策	选择	理由
识别方式	特征匹配（非分类）	新增学生无需重训分类头，只需提取特征向量入库
人脸检测	MTCNN	多尺度检测，比 Haar Cascade 准确率高 ~15%
特征维度	512d（VGG16）/ 归一化后 512d（ResNet50 avg pool）	平衡精度与匹配速度
特征存储	numpy `.npy` 文件 + 内存加载	百人级别匹配 <1ms，无需向量数据库
实时流处理	WebSocket 推送识别结果	避免前端轮询，延迟更低
模型对比评估指标	Top-1 准确率、F1 Score、ROC-AUC、推理时间	论文实验对比章节需要

---

7. 数据库索引设计

合理索引是保证查询性能的关键。以下为各表核心索引规划:

表名	索引名	字段	类型	用途
users	uk_username	username	UNIQUE	登录账号唯一性校验
students	uk_student_id	student_id	UNIQUE	学号唯一性，考勤写库时用
students	idx_class	class_name	INDEX	按班级查询学生列表
courses	uk_course_code	course_code	UNIQUE	课程编号唯一性
courses	idx_teacher	teacher_id	INDEX	教师查自己的课程
student_courses	uk_enroll	(student_id, course_id, semester)	UNIQUE	防重复选课
attendance_tasks	idx_course_date	(course_id, task_date)	INDEX	按课程 + 日期查考勤任务
attendance_tasks	idx_teacher	teacher_id	INDEX	教师查自己发起的任务
attendance_tasks	idx_status	status	INDEX	按状态过滤任务
attendance_records	uk_task_student	(task_id, student_id)	UNIQUE	同任务同学生只一条记录
attendance_records	idx_student	student_id	INDEX	学生查自己的考勤历史
attendance_records	idx_status	status	INDEX	过滤待核实、缺勤
face_images	idx_student	student_id	INDEX	查学生的人脸图片列表

---

8. 部署架构设计

8.1 开发环境

```
开发机（本地）
  Frontend Dev Server  :5173  (Vite + Hot Reload)
  Backend Flask Dev    :5000  (Flask run + debug mode)
  MySQL                :3306  (本地 MySQL 8.0)
  模型文件              ./model/checkpoints/
```

8.2 生产/演示环境（单机部署）

```
服务器（Linux / Ubuntu 22.04，推荐 8C16G，可选 GPU）

  Nginx（反向代理 + 静态文件服务）  :80 / :443
   /        → 前端 dist/（Vue.js 编译产物）
   /api      → 代理至 Gunicorn :5000
   /ws       → 代理 WebSocket 至 :5001

 Gunicorn（4 Worker）      MySQL 8.0（本机实例）
  Flask App :5000          attendance_db

  文件系统
   /data/faces/{student_id}/   人脸图片
   /data/checkpoints/          模型权重
   /data/features.npy          特征数据库
```

8.3 关键配置参数

参数	推荐值	说明
Gunicorn workers	4	CPU 核数 ×1，I/O 密集型
Gunicorn timeout	120s	模型推理任务不超时
MySQL max_connections	200	并发连接数
Nginx client_max_body_size	20m	人脸图片上传限制
Nginx proxy_read_timeout	300s	模型训练接口超时

---

9. WebSocket 通信协议设计

考勤任务的实时识别结果通过 WebSocket 推送，避免前端轮询。

9.1 连接建立

```
URL:  ws://<host>/ws/attendance/<task_id>
Headers:
  Authorization: Bearer <JWT Token>
```

9.2 消息格式（JSON）

前端 → 后端（发送摄像头帧）：

```
{
  "type": "frame",
  "task_id": 42,
  "image": "<base64 encoded JPEG>",
  "timestamp": 1713236400.123
}
```

后端 → 前端（推送识别结果）：

```
{
  "type": "recognition_result",
  "task_id": 42,
  "results": [
    {
      "student_id": "2022214380",
      "student_name": " 苏东煌",
```

```
      "status": "present",
      "confidence": 0.923,
      "bbox": [120, 85, 220, 210]
    }
  ],
  "timestamp": 1713236400.456,
  "frame_stats": {
    "faces_detected": 2,
    "recognized": 1,
    "unverified": 1
  }
}
```

后端 → 前端（任务状态更新）：

```
{
  "type": "task_update",
  "task_id": 42,
  "status": "finished",
  "summary": {
    "total_students": 35,
    "present_count": 30,
    "absent_count": 5,
    "unverified_count": 2,
    "attendance_rate": 0.857
  }
}
```

后端 → 前端（错误通知）：

```
{
  "type": "error",
  "code": "MODEL_NOT_LOADED",
  "message": " 识别模型尚未加载，请联系管理员"
}
```

9.3 帧率控制策略

- 前端默认每 **500ms** 推送一帧（2 fps）
- 若上一帧尚未处理完（后端繁忙），前端自动跳过本帧，避免积压
- 教室人数 30 时，推理延迟目标 200ms/帧

---

10. 错误处理策略

10.1 HTTP 错误码规范

HTTP 状态码	业务场景	前端处理建议
400	请求参数校验失败	表单校验提示，显示具体字段错误
401	Token 缺失或过期	清除本地 Token，跳转登录页
403	权限不足	提示“无权限操作”，不跳转
404	资源不存在	提示“未找到相关数据”
409	业务冲突（如学号重复）	提示具体冲突原因
422	数据格式正确但业务校验失败	显示后端返回的 message 字段
500	服务器内部错误	提示“服务异常，请联系管理员”，上报错误码

10.2 AI 模块异常处理

异常场景	处理方式	降级策略
模型文件缺失	启动时检测，缺失则记录警告日志	返回 503，提示“模型未就绪”
推理超时（>5s）	超时中断，返回空结果	标记本帧为 unverified
显存不足（GPU OOM）	捕获 CUDA 异常，自动切换 CPU 模式	CPU 推理，性能下降但不中断
人脸检测返回空	正常返回空列表	前端提示“当前帧未检测到人脸”
特征库为空	考勤开始前校验	阻止任务启动，提示“该课程暂无已注册学生”

10.3 统一异常响应结构

```
# 后端统一错误响应格式
{
    "code": 400,
    "message": "student_id: 该学号已存在",
    "data": null,
    "request_id": "req-abc123"    # 便于日志追踪
}
```